Introduction(2).pdf
PDF

給我第一頁到第五頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第1頁到第5頁逐字英文→中文翻譯（完全對應，不增不減）**：

第1頁

Introduction

CHAPTER 1
第1章

BASIC CONCEPTS
基本概念

Jai-Ming Lin Spring 2026
林宰明 2026年春季

第2頁

Introduction

Basic Concepts
基本概念

Basic Concepts Introduction to Data Structure P2
基本概念 資料結構導論 第2頁

□ Contents
□ 內容

□ System life cycle
□ 系統生命週期

□ Object-oriented design
□ 物件導向設計

□ Data abstraction and encapsulation
□ 資料抽象與封裝

□ Algorithm specification
□ 演算法規格

□ Performance analysis and measurement
□ 效能分析與測量

□ Readings
□ 閱讀

□ Chapter 1
□ 第1章

□ C++
□ C++

■ H. Deitel and P. Deitel, C++ How to Program, 5th Ed., Prentice Hall, 2005. ISBN: 0131971093.
■ H. Deitel 和 P. Deitel，《C++ How to Program》，第5版，Prentice Hall，2005年。ISBN：0131971093。

第3頁

Introduction

System Life Cycle
系統生命週期

□ System life cycle: refer to programs undergo a development process if large-scale computer programs are regarded as systems
□ 系統生命週期：如果將大型電腦程式視為系統，則指程式會經歷一個開發過程

□ The cycle contains of the five phases listed in the following:
□ 此週期包含以下列出的五個階段：

1. Requirements

1. 需求

□ Define specifications (input and output): vague □ rigorous
□ 定義規格（輸入與輸出）：模糊 → 嚴謹

2. Analysis
2. 分析

□ Divide into manageable pieces: top-down vs. bottom-up
□ 分割成可管理的部分：由上而下 vs. 由下而上

■ Bottom-up is an older, unstructured strategy that placesan early emphasis on the coding fine points.
■ 由下而上是一種較舊、非結構化的策略，早期就強調程式碼的細節

■ Top-down begins by developing a high-level plan for dividing the program into manageable segments, and is then refinedto take into account low-level details.
■ 由上而下從建立高階計畫開始，將程式分成可管理的部分，然後再細化以考慮低階細節

preferred
較佳

Hierarchical approach
階層式方法

Basic Concepts Introduction to Data Structure P3
基本概念 資料結構導論 第3頁

第4頁

Introduction

System Life Cycle
系統生命週期

3. Design (language-independent)
3. 設計（與語言無關）

□ Approach the system from the data objects that program needs and operations performed on them
□ 從程式所需的資料物件以及對它們執行的操作來設計系統

■ The first part is creation of abstract data types
■ 第一部分是建立抽象資料型態

■ The second requires the specification of algorithms and their strategies.
■ 第二部分需要定義演算法及其策略

 The advantage includes creating a system that fit in several programming languages and having time to pick the most efficient implementations in the chosen language.
 優點包括建立可適用於多種程式語言的系統，並有時間在所選語言中選擇最有效率的實作

3. Refinement and coding (language dependent)
3. 細化與編碼（與語言相關）

 Choose representations for our data objects and write algorithms for each operation
 為資料物件選擇表示方式並為每個操作撰寫演算法

5. Verification
5. 驗證

 Correctness proofs: use formal (mathematics) techniques
 正確性證明：使用形式化（數學）方法

 Testing: include all possible scenarios; estimate performance
 測試：包含所有可能情境；估計效能

 Error removal: debug based on above two; avoid spaghetti codes
 錯誤排除：依據上述兩者進行除錯；避免義大利麵式程式碼

Hierarchical approach
階層式方法

Basic Concepts Introduction to Data Structure P4
基本概念 資料結構導論 第4頁

第5頁

Introduction

5 Object-Oriented Design
5 物件導向設計

Basic Concepts
基本概念

Object-Oriented Design
物件導向設計

如果你下一步要：
👉「逐頁詳細講解（老師會考的重點）」
👉「幫你整理成考試筆記 / 心智圖」

我可以直接幫你做一版🔥

Sources

給我第6頁到第10頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第6頁到第10頁逐字英文→中文翻譯（完全對應，不增不減）**：

第6頁

Introduction

Algorithmic vs. OO Decomposition
演算法式 vs. 物件導向分解

□ Algorithmic (functional) decomposition views software as a process
□ 演算法（功能式）分解將軟體視為一個過程

□ Decompose the software into modules that represent steps of the process
□ 將軟體分解為代表過程步驟的模組

□ Data structures are a secondary concern
□ 資料結構是次要考量

□ Object-Oriented decomposition views software as a set of well- defined objects that model entities in the applicationdomain
□ 物件導向分解將軟體視為一組良好定義的物件，用來模擬應用領域中的實體

□ Interact with each other to form a software system
□ 彼此互動以形成一個軟體系統

□ Functional decomposition is addressed after the system has been decomposed into objects
□ 功能分解是在系統被分解成物件之後才處理

□ Advantages of OO decomposition:
□ 物件導向分解的優點：

□ Encourage reuse of software
□ 鼓勵軟體重複使用

□ Allow software to evolve as system requirements change
□ 允許軟體隨著系統需求改變而演進

□ Is more intuitive: objects naturally model entities in the application domain
□ 更直觀：物件自然地對應應用領域中的實體

Divide-and-Conquer
分而治之

Basic Concepts Introduction to Data Structure P6
基本概念 資料結構導論 第6頁

第7頁

Introduction

Object-Oriented Programming
物件導向程式設計

Basic Concepts Introduction to Data Structure P7
基本概念 資料結構導論 第7頁

□ Definition: An object is an entity that performscomputations and has a local state
□ 定義：物件是一個執行計算並具有區域狀態的實體

□ May be viewed as a combination of data and procedural elements
□ 可以被視為資料與程序元素的組合

□ Definition: Object-oriented programming is a methodof implementation in which
□ 定義：物件導向程式設計是一種實作方法，其中

1. Objects are the fundamental building blocks

1. 物件是基本的建構單位

2. Each object is an instance of some type (or class)

2. 每個物件都是某種類型（或類別）的實例

3. Classes are related to each other by inheritance relationships

3. 類別彼此之間透過繼承關係相連

□ Definition: An object-orientedlanguage

□ 定義：物件導向語言

1. Supports objects

1. 支援物件

2. Requires objects to belong to a class

2. 要求物件屬於某個類別

3. Supports inheritance

3. 支援繼承

□ The first two features are considered as object-based

□ 前兩個特性被視為物件基礎

第8頁

Introduction

8 Data Abstraction and Encapsulation

8 資料抽象與封裝

Basic Concepts

基本概念

Data Abstraction and Encapsulation

資料抽象與封裝

第9頁

Introduction

Example: DVD Players
範例：DVD 播放器

Basic Concepts Introduction to Data Structure P9
基本概念 資料結構導論 第9頁

□ Packaging: Control panel (remote control) of a DVD player:
□ 封裝：DVD 播放器的控制面板（遙控器）：

□ Interact only through buttons
□ 僅透過按鈕互動

□ Encapsulation: hide internal representation from users
□ 封裝：對使用者隱藏內部表示

□ Usage: Instruction manual of the DVD player
□ 使用：DVD 播放器的使用手冊

□ Does tell us what the DVD player does if a button is pressed
□ 告訴我們按下按鈕時 DVD 播放器會做什麼

□ Does not tell us tell us how the function is implemented
□ 不會告訴我們功能是如何實作的

□ Abstraction: describe what to do, not how to do
□ 抽象：描述做什麼，而不是如何做

第10頁

Introduction

Data Abstraction & Encapsulation implementation
資料抽象與封裝 實作

□ Definition: Data encapsulation (or informationhiding)
□ 定義：資料封裝（或資訊隱藏）

□ Conceal the implementation details of a data object from the outside world
□ 將資料物件的實作細節對外界隱藏

Make a data object as a black box
將資料物件視為黑盒子

□ **Definition: Data abstraction**
□ 定義：資料抽象

□ **Separate the specification of a data object and its**
□ 將資料物件的規格與其分離

What to do vs. how to do
做什麼 vs. 如何做

Basic Concepts Introduction to Data Structure P10
基本概念 資料結構導論 第10頁

如果你下一步要：
👉「這幾頁幫你整理考試重點（超重要🔥）」
👉「OOP、抽象、封裝一張圖講懂」

我可以幫你直接整理一版「會考版本」👍

Sources

Introduction(3).pdf
PDF

給我第11頁到第15頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第11頁到第15頁逐字英文→中文翻譯（完全對應，不增不減）**：

第11頁

Introduction

Data Type
資料型態

Basic Concepts Introduction to Data Structure P11
基本概念 資料結構導論 第11頁

□ Definition: A data type is
□ 定義：資料型態是

1. A collection of objects, and

1. 一組物件的集合，且

2. A set of operations that act on thoseobjects
2. 一組作用於這些物件的操作

□ Example: int
□ 範例：int

1. Objects: {0, +1, -1, +2, -2, ..., MAXINT, MININT}

1. 物件：{0, +1, -1, +2, -2, ..., MAXINT, MININT}

■ MAXINT: maximum integer on the computer
■ MAXINT：電腦中的最大整數

■ MININT: minimum
■ MININT：最小值

**2. Operations: {+, - , , /, %, <<, >>, ==, !=,...}*
2. 操作：{+, - , *, /, %, <<, >>, ==, !=,...}

第12頁

Introduction

Abstract Data Type (ADT)
抽象資料型態（ADT）

□ Definition: An abstract data type (ADT) is a data type thatis organized such that
□ 定義：抽象資料型態（ADT）是一種資料型態，其被組織成使得

□ The specification of the objects is separated from the representation of the objects
□ 物件的規格與物件的表示分離

□ The specification of the operations on the objects is separated from the implementation of the operations

□ 對物件的操作規格與操作的實作分離

ADT = abstraction + data type
ADT = 抽象 + 資料型態

□ Separate specification from implementation:
□ 將規格與實作分離：

A specification can have several options of implementation
一個規格可以有多種實作方式

Basic Concepts Introduction to Data Structure P12
基本概念 資料結構導論 第12頁

第13頁

Introduction

ADT Example: NaturalNumber
ADT 範例：NaturalNumber

ADT NaturalNumber is
ADT NaturalNumber 是

objects:An ordered subrange of the integers starting at 0, ending at MAXINT on the computer.
物件：從0開始，到電腦中的MAXINT結束的整數有序子範圍。

functions:
函式：

for all x, y □ NaturalNumber; TRUE, FALSE □ Boolean
對所有 x, y ∈ NaturalNumber；TRUE, FALSE ∈ Boolean

and where +, -, <, ==, and = are the usual integer operations
且其中 +, -, <, ==, 和 = 是一般的整數運算

0
0

if (x == 0) IsZero =TRUE
如果 (x == 0) IsZero = TRUE

else IsZero = FALSE
否則 IsZero = FALSE

if (x+y <= MAXINT) Add = x +y
如果 (x+y <= MAXINT) Add = x + y

else Add = MAXINT
否則 Add = MAXINT

if (x == y) Equal =TRUE
如果 (x == y) Equal = TRUE

else Equal = FALSE
否則 Equal = FALSE

if (x == MAXINT) Successor =x
如果 (x == MAXINT) Successor = x

else Successor = x +1
否則 Successor = x + 1

if (x < y) Substract = 0
如果 (x < y) Substract = 0

else Substract = x – y
否則 Substract = x – y

Zero(): NaturalNumber ::=
Zero(): NaturalNumber ::=

IsZero(x): Boolean ::=
IsZero(x): Boolean ::=

Add(x, y): NaturalNumber ::=
Add(x, y): NaturalNumber ::=

Equal(x, y): Boolean ::=
Equal(x, y): Boolean ::=

Successor(x): NaturalNumber ::=
Successor(x): NaturalNumber ::=

Substract(x, y): NaturalNumber ::=
Substract(x, y): NaturalNumber ::=

end NaturalNumber
結束 NaturalNumber

::= is defined as
::= 定義為

Later we use the syntax of C++ class to express an ADT
之後我們會使用 C++ 類別的語法來表達 ADT

Basic Concepts Introduction to Data Structure P13
基本概念 資料結構導論 第13頁

第14頁

Introduction

Pros of Data Abstraction & Encapsulation
資料抽象與封裝的優點

□ Simplification of software development (data abs)
□ 軟體開發的簡化（資料抽象）

□ Facilitate the decomposition of a complex task into simpler ones
□ 促進將複雜任務分解為較簡單的任務

■ Scenario 1- Ateam of four programmers: Do not need to know how the other programmers implement their portion of the codes
■ 情境1 - 四位程式設計師的團隊：不需要知道其他程式設計師如何實作他們的程式碼部分

■ Scenario 2- Asingle programmer: Reduce the number of things the programmer has to keep in mind at any time
■ 情境2 - 單一程式設計師：減少程式設計師在任何時間需要記住的事情數量

□ Testing and debugging (data abs)
□ 測試與除錯（資料抽象）

□ Lead to more efficient testing and debugging.
□ 導致更有效率的測試與除錯

■ Facilitate test and debug separately; otherwise, the amount of code has to be searched otherwise.
■ 促進分開測試與除錯；否則必須搜尋的程式碼量會很大

A
A

Glue
膠合

B
B

C
C

Abstraction Or not
抽象 或 否

Basic Concepts Introduction to Data Structure P14
基本概念 資料結構導論 第14頁

第15頁

Introduction

Pros of Data Abstraction & Encapsulation
資料抽象與封裝的優點

Basic Concepts Introduction to Data Structure P15
基本概念 資料結構導論 第15頁

□ Reusability (data abs & enc)
□ 可重用性（資料抽象與封裝）

□ Make it easier to extract the code for a data structure and its operations from a software system and use it in another system
□ 使得從一個軟體系統中抽取資料結構及其操作的程式碼並在另一個系統中使用變得更容易

□ Modifications to the representation of a data type (data enc)
□ 資料型態表示的修改（資料封裝）

□ Implementation of a data type is invisible to the outside world
□ 資料型態的實作對外界是不可見的

■ A change in the internal implementation of a data type will not affect the rest of program if the data types provides the same operations and the operations do the same thing.

■ 如果資料型態提供相同的操作且操作執行相同的事情，則其內部實作的改變不會影響程式的其他部分

□ What if without data enc?

□ 如果沒有資料封裝會怎樣？

如果你要下一步：
👉「這5頁整理成考試必背（老師愛考🔥）」
👉「ADT / encapsulation 一張圖講懂」

我可以直接幫你整理一版超好記的版本 👍

Sources

給我第16頁到第20頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第16頁到第20頁逐字英文→中文翻譯（完全對應，不增不減）**：

第16頁

Introduction

Algorithm Specification
演算法規格

Basic Concepts
基本概念

16
16

第17頁

Introduction

Algorithm
演算法

□ Definition: An algorithm is
□ 定義：演算法是

□ A finite set of instructions that accomplishes a particular task
□ 一組有限的指令集合，用來完成特定任務

□ Criteria:
□ 準則：

1. Input: may have

1. 輸入：可以有

2. Output: must have
2. 輸出：必須有

Task (problem)
任務（問題）

Procedure (solution)
程序（解決方案）

Problem-solving procedure
解決問題的程序

3. Definiteness: (every instruction) must be clear and unambiguous
3. 明確性：（每個指令）必須清楚且不模糊

4. Finiteness: terminate after a finite number of steps
4. 有限性：在有限步驟後終止

5. Effectiveness: (every instruction) must be basic and feasible with pencil and paper
5. 有效性：（每個指令）必須是基本且可以用紙筆完成

□ Cf. An algorithm is
□ 參考：演算法是

□ A well-defined procedure for transforming some input to a desired output [Cormen et al. Introduction to Algorithms, 2ndEd.]

 一個良好定義的程序，用來將某些輸入轉換為期望的輸出 [Cormen 等，《Introduction to Algorithms》，第2版]

** A well-defined procedure to solve a problem; a programmerturns the algorithm into a computer program [Sci-Tech Encyclopedia]**
 一個良好定義的程序來解決問題；程式設計師將演算法轉換為電腦程式 [Sci-Tech Encyclopedia]

high-level low-level
高階 低階

Basic Concepts Introduction to Data Structure P17
基本概念 資料結構導論 第17頁

第18頁

Introduction

Problem
問題

Algorithm
演算法

Program
程式

Basic Concepts Introduction to Data Structure P18
基本概念 資料結構導論 第18頁

Recap System Life Cycle
系統生命週期回顧

□ Regard a large-scale computer program as a system
□ 將大型電腦程式視為一個系統

** Programming >> coding**
  程式設計 >> 編碼

1. Requirements

1. 需求

□ Define specifications (input and output): vague □

□ 定義規格（輸入與輸出）：模糊 →

2. rigorous Analysis

2. 嚴謹 分析

□ Divide into manageable pieces: top-down vs. bottom-up

□ 分割成可管理的部分：由上而下 vs. 由下而上

3. Design

3. 設計

□ Determine data objects (abstract data types) and operations

□ 決定資料物件（抽象資料型態）與操作

4. Refinement and (algorithms): language coding-independent

4. 細化與（演算法）：與語言無關的編碼

□ Implement: language□5. dependent Verification

□ 實作：與語言相關 5. 驗證

□ Correctness proofs: use formal (mathematics) techniques

□ 正確性證明：使用形式化（數學）方法

□ Testing: include all possible scenarios; estimate performance

□ 測試：包含所有可能情境；估計效能

□ Error removal: debug based on above two; avoid spaghetti codes

□ 錯誤排除：依據上述兩者進行除錯；避免義大利麵式程式碼

第19頁

Introduction

Selection Sort

選擇排序

□ Problem: Sorting

□ 問題：排序

□ Devise a program that sorts a collection of n □ 1 integers

□ 設計一個程式來排序一組 n ≥ 1 的整數

□ Input: a sequence of n integers <a1, a2, ..., an>, n □1
□ 輸入：一個 n 個整數的序列 <a1, a2, ..., an>，n ≥ 1

□ Output: a permutation <a'1, a'2, ..., a'n> s.t. a'1 □ a'2 □ ... □ a'n
□ 輸出：一個排列 <a'1, a'2, ..., a'n> 使得 a'1 ≤ a'2 ≤ ... ≤ a'n

□ Solution: Selection sort
□ 解法：選擇排序

□ From those integers that are currently unsorted, find the smallestand place it next in the sorted list. [in English]
□ 從目前尚未排序的整數中找出最小值，並將其放到已排序列表的下一個位置［英文描述］

□ Well-defined? Clear? Unambiguous?
□ 是否定義良好？清楚？無歧義？

■ Where and how are the integers initially stored?
■ 整數最初儲存在何處以及如何儲存？

■ Where to place the results?
■ 將結果放在哪裡？

System life cycle: analysis & design
系統生命週期：分析與設計

System life cycle: requirements
系統生命週期：需求

Basic Concepts Introduction to Data Structure 3/5/202P619
基本概念 資料結構導論 3/5/202P619

第20頁

Introduction

Introduction to Data Structure
資料結構導論

Selection Sort - Algorithm & Program
選擇排序 - 演算法與程式

**void SelectionSort (int a, const int n)*
void SelectionSort (int *a, const int n)

{// Sort the n integers a[0] to a[n-1]into nondecreasing order
{// 將 n 個整數 a[0] 到 a[n-1] 排序為非遞減順序

for (int i = 0; i < n; i++)
for (int i = 0; i < n; i++)

{
{

examine a[i] to a[n-1] and suppose the smallest integer is at a[j];
檢查 a[i] 到 a[n-1]，並假設最小整數位於 a[j]；

interchange a[i] and a[j];
交換 a[i] 和 a[j]；

}
}

}
}

**void SelectionSort (int a, const int n)*
void SelectionSort (int *a, const int n)

{// Sort the n integers a[0] to a[n-1]into nondecreasing order
{// 將 n 個整數 a[0] 到 a[n-1] 排序為非遞減順序

for (int i = 0; i < n; i++)
for (int i = 0; i < n; i++)

{
{

int j = i;
int j = i;

// find smallest integer in a[i] to a[n-1]
// 在 a[i] 到 a[n-1] 中找最小整數

for (int k = i + 1; k < n; k++)
for (int k = i + 1; k < n; k++)

if (a[k] < a[j]) j = k;
如果 (a[k] < a[j]) 則 j = k;

swap(a[i], a[j]);
swap(a[i], a[j]);

}
}

}
}

Algorithm Program
演算法 程式

Basic Concepts P20
基本概念 第20頁

Theorem: SelectionSort(a, n) correctly sorts a set of n □ 1 integers; the result remains in a[0] … a[n-1] such that a[0] □ a[1] □ … □ a[n–1].
定理：SelectionSort(a, n) 能正確地排序一組 n ≥ 1 的整數；結果保存在 a[0] … a[n-1] 中，使得 a[0] ≤ a[1] ≤ … ≤ a[n–1]

Q: How to prove it?
問題：如何證明？

System life cycle: refinement & coding
系統生命週期：細化與編碼

System life cycle: analysis & design
系統生命週期：分析與設計

High-level Language independent
高階 與語言無關

Low-level Language dependent
低階 與語言相關

如果你要下一步：
👉「這幾頁幫你整理成考試必背（超重要🔥）」
👉「Selection Sort 一次讓你完全懂（含考題）」

我可以直接幫你做🔥

Sources

給我第21頁到第25頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第21頁到第25頁逐字英文→中文翻譯（完全對應，不增不減）**：

第21頁

Introduction

Binary Search
二元搜尋

□ Problem: Searching
□ 問題：搜尋

□ Input
□ 輸入

■ a sorted array of n □ 1 distinct integers a[0..n-1]
■ 一個已排序的包含 n ≥ 1 個不同整數的陣列 a[0..n-1]

■ integer x
■ 整數 x

□ Output
□ 輸出

■ return j, if there exists j such that x = a[j]
■ 如果存在 j 使得 x = a[j]，則回傳 j

■ return -1, otherwise
■ 否則回傳 -1

□ Solution:
□ 解法：

□ Compare one by one: Sequential search
□ 一個一個比較：循序搜尋

■ Correct but slow
■ 正確但緩慢

□ Better idea?
□ 更好的方法？

■ Hint: input is sorted
■ 提示：輸入是已排序的

■ Divide-and-conquer: Binary search
■ 分而治之：二元搜尋

Use known information to improve your solution
利用已知資訊來改善你的解法

Basic Concepts Introduction to Data Structure P21
基本概念 資料結構導論 第21頁

第22頁

Introduction

Binary Search - Divide-and-Conquer
二元搜尋 - 分而治之

Basic Concepts Introduction to Data Structure P22
基本概念 資料結構導論 第22頁

□ Divide the problem into a number of subproblems
□ 將問題分成多個子問題

□ Similar but easier
□ 類似但較簡單

□ Conquer the subproblems
□ 解決子問題

□ Solve them
□ 解決它們

□ Combine the subsolutions to get the solution to the original problem
□ 將子解組合以得到原問題的解

□ Divide: check the middle element
□ 分割：檢查中間元素

□ Conquer: search the subarray
□ 解決：搜尋子陣列

□ Combine: trivial
□ 合併：簡單

Divide-and-conquer paradigm Binary search on a sorted array
分而治之典範 已排序陣列上的二元搜尋

0 5 13 19 22 41 55 68 72 81 98 55
0 5 13 19 22 41 55 68 72 81 98 55

55 68 72 81 98 55
55 68 72 81 98 55

55 68 55
55 68 55

< > =
< > =

第23頁

Introduction

Binary Search- Algorithm & Program
二元搜尋 - 演算法與程式

Basic Concepts Introduction to Data Structure P23
基本概念 資料結構導論 第23頁

**int BinarySearch (int a, const int x, constint n)*
int BinarySearch (int *a, const int x, const int n)

{// Search the sorted array a[0], … , a[n-1] forx
{// 在排序好的陣列 a[0], … , a[n-1] 中搜尋 x

initialize left and right;
初始化 left 和 right；

while (there are more elements)
當（仍有元素）

{
{

let middle be the middle element;
令 middle 為中間元素；

if (x < a[middle]) set right to middle-1;
如果 (x < a[middle]) 將 right 設為 middle-1；

else if (x > a[middle]) set left to middle+1;
否則如果 (x > a[middle]) 將 left 設為 middle+1；

else return middle;
否則回傳 middle；

}
}

return -1; //not found
回傳 -1； //未找到

}
}

**int BinarySearch (int a, const int x, constint n)*
int BinarySearch (int *a, const int x, const int n)

{// Search the sorted array a[0], ... , a[n-1] forx
{// 在排序好的陣列 a[0], ... , a[n-1] 中搜尋 x

int left = 0, right = n-1;
int left = 0, right = n-1;

while (left <= right)
當 (left <= right)

{// there are more elements
{// 仍有元素

int middle = (left + right)/2;
int middle = (left + right)/2;

if (x < a[middle]) right = middle-1;
如果 (x < a[middle]) right = middle-1；

else if (x > a[middle]) left = middle+1;
否則如果 (x > a[middle]) left = middle+1；

else return middle;
否則回傳 middle；

} //end of while
} //while 結束

return -1; //not found
回傳 -1； //未找到

}
}

Algorithm Program (Iterative)
演算法 程式（迭代）

0 5 13 19 22 41 55 68 72 81 98
0 5 13 19 22 41 55 68 72 81 98

left middle right
left middle right

55 68 72 81 98
55 68 72 81 98

left middle right
left middle right

55 68 55
55 68 55

a[middle]==55, done!
a[middle]==55，完成！

left right
left right

< 55 > 55 =

< 55 > 55 =

第24頁

Introduction

RecursiveAlgorithms

遞迴演算法

□ Definition: A recursive functionis

□ 定義：遞迴函式是

 A function that invokes itself before it is done

 一個在完成之前會呼叫自己的函式

■ Direct (call by itself) vs. indirect (call through others)

■ 直接（自己呼叫自己）vs. 間接（透過其他呼叫）

□ Why to use recursion?

□ 為什麼使用遞迴？

 Often can express a complex process very clearly

 通常可以非常清楚地表達複雜過程

□ Any program that can be written using assignment+if-else+while can be written using assignment+if-else+recursion

□ 任何可以用 指派+if-else+while 寫出的程式，也可以用 指派+if-else+遞迴 寫出

 Question: When to use recursion?

 問題：何時使用遞迴？

 Answer: The problem itself is recursively defined

 答案：當問題本身是遞迴定義時

■ e.g., factorials n!, Fibonacci sequence Fn = Fn-1 + Fn-2

■ 例如：階乘 n!，費波那契數列 Fn = Fn-1 + Fn-2

■ e.g., divide-and-conquer approach

■ 例如：分而治之方法

Mathematical induction

數學歸納法

Basic Concepts Introduction to Data Structure P24
基本概念 資料結構導論 第24頁

第25頁

Introduction

RecursiveAlgorithms
遞迴演算法

□ To understand a recursive function, you must
□ 要理解一個遞迴函式，你必須

1. Realize what it is that the function is supposed to do

1. 理解該函式應該做什麼

2. Verify correctness of function if the recursive invocations to itself
2. 驗證當函式遞迴呼叫自身時的正確性

3. Ensure a finite number of recursive invocations will lead to an invocation that satisfies the terminating condition
3. 確保有限次的遞迴呼叫會導致一個滿足終止條件的呼叫

4. Perform the correct computations if the terminal condition happens
4. 當終止條件發生時執行正確的計算

□ How to develop recursion?
□ 如何發展遞迴？

1. Terminating condition (basis)

1. 終止條件（基礎）

■ Trivial cases; directly computes the output
■ 簡單情況；直接計算輸出

2. Recursion (induction hypothesis)
2. 遞迴（歸納假設）

■ If n=k-1, the statement holds, checkn=k?
■ 如果 n=k-1 時命題成立，檢查 n=k？

Mathematical induction
數學歸納法

Basic Concepts Introduction to Data Structure P25
基本概念 資料結構導論 第25頁

如果你要下一步：
👉「Binary Search + Recursion 一次幫你完全理解（考試重點🔥）」
👉「幫你整理成一頁 cheat sheet」

我可以直接幫你做 👍

Sources

給我第26頁到第30頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第26頁到第30頁逐字英文→中文翻譯（完全對應，不增不減）**：

第26頁

Introduction

Binary Search - Recursion
二元搜尋 - 遞迴

Start with BinarySearch(a, x, 0, n-1)
從 BinarySearch(a, x, 0, n-1) 開始

Faster
較快

Program (Iterative)
程式（迭代）

**int BinarySearch (int a, const int x, constint n)*
int BinarySearch (int *a, const int x, const int n)

{// Search the sorted array a[0], … , a[n-1] forx
{// 在排序好的陣列 a[0], … , a[n-1] 中搜尋 x

int left = 0, right = n-1;
int left = 0, right = n-1;

while (left <= right)
當 (left <= right)

{// there are more elements
{// 仍有元素

int middle = (left + right)/2;
int middle = (left + right)/2;

if (x < a[middle]) right = middle-1;
如果 (x < a[middle]) right = middle-1；

else if (x > a[middle]) left = middle+1;
否則如果 (x > a[middle]) left = middle+1；

else return middle;
否則回傳 middle；

} //end of while
} //while 結束

return -1; //not found
回傳 -1； //未找到

}
}

More intuitive and elegant
更直觀且優雅

Program (Recursive)
程式（遞迴）

**int BinarySearch (int a, const int x, constint left, const int right)*
int BinarySearch (int *a, const int x, const int left, const int right)

{// Search the sorted array a[left], … , a[right] forx
{// 在排序好的陣列 a[left], … , a[right] 中搜尋 x

if (left <= right) {
如果 (left <= right) {

int middle = (left + right)/2;
int middle = (left + right)/2;

if (x < a[middle]) {
如果 (x < a[middle]) {

return BinarySearch(a, x, left, middle-1);
回傳 BinarySearch(a, x, left, middle-1);

} else if (x > a[middle]) {
} 否則如果 (x > a[middle]) {

return BinarySearch(a, x, middle+1, right);
回傳 BinarySearch(a, x, middle+1, right);

} else return middle;
} 否則回傳 middle；

} //end of if
} //if 結束

return -1; //not found
回傳 -1； //未找到

}
}

Basic Concepts Introduction to Data Structure P26
基本概念 資料結構導論 第26頁

第27頁

Introduction

Permutation Generator
排列產生器

Basic Concepts Introduction to Data Structure P27
基本概念 資料結構導論 第27頁

□ **Problem:**
□ 問題：

 Input: a set of n 1 elements
 輸入：一組 n ≥ 1 個元素

 Output: all possible permutations of this set
 輸出：此集合的所有可能排列

 e.g., input: {a,b,c} output: {(a,b,c), (a,c,b), (b,a,c), (b,c,a), (c,a,b), (c,b,a)}
 例如，輸入：{a,b,c} ⇒ 輸出：{(a,b,c), (a,c,b), (b,a,c), (b,c,a), (c,a,b), (c,b,a)}

□ **Solution:**
□ 解法：

 Given {a,b,c}, the answer can be constructed by writing
 給定 {a,b,c}，答案可以透過以下方式構造

■ **a followed by all permutations of {b, c}**
■ a 接上 {b, c} 的所有排列

■ **b followed by all permutations of {a, c}**
■ b 接上 {a, c} 的所有排列

■ **c followed by all permutations of {a, b}**
■ c 接上 {a, b} 的所有排列

 If we can solve n-1 elements, we then can solve n elements
 如果我們能解決 n-1 個元素，就能解決 n 個元素

第28頁

Introduction

Permutation Generator - Recursion
排列產生器 - 遞迴

**void Permutations (char a, const int k, const int m)*
void Permutations (char *a, const int k, const int m)

{// Generate all the permutations of a[k], ..., a[m]
{// 產生 a[k], ..., a[m] 的所有排列

if (k == m) { // output permutation
如果 (k == m) { // 輸出排列

for (int i = 0; i <= m; i++) cout << a[i] <<" ";
for (int i = 0; i <= m; i++) cout << a[i] << " ";

cout << endl;
cout << endl;

}
}

else // a[k:m] has more than one permutation. Generate these recursively.
否則 // a[k:m] 有多個排列。以遞迴方式產生

for (i = k; i <= m; i++)) {
for (i = k; i <= m; i++)) {

swap(a[k], a[i]);
swap(a[k], a[i]);

Permutations(a, k+1, m);
Permutations(a, k+1, m);

swap(a[k], a[i]);
swap(a[k], a[i]);

}
}

}
}

Start with Permutations(a, 0, n-1)
從 Permutations(a, 0, n-1) 開始

Basic Concepts Introduction to Data Structure P28
基本概念 資料結構導論 第28頁

第29頁

Introduction

29 Performance Analysis and Measurement
29 效能分析與測量

Basic Concepts
基本概念

Performance Analysis and Measurement
效能分析與測量

第30頁

Introduction

Complexity
複雜度

Basic Concepts Introduction to Data Structure P30
基本概念 資料結構導論 第30頁

□ Definition: The space complexity of a programis
□ 定義：程式的空間複雜度是

□ The amount of memory it needs
□ 它所需的記憶體量

□ □ Storage requirements
□ ⇒ 儲存需求

□ Definition: The time complexity of a program is
□ 定義：程式的時間複雜度是

□ The amount of computer time it needs
□ 它所需的電腦時間

□ □ Computing time or runtime
□ ⇒ 計算時間或執行時間

□ Performance evaluation
□ 效能評估

□ Performance analysis: priori estimates
□ 效能分析：事前估計

□ Performance measurement: posteriori testing
□ 效能測量：事後測試

如果你要下一步：
👉「Binary Search vs Recursion vs Iteration 考試重點整理🔥」
👉「Permutation + Recursion 一次讓你完全懂（含圖解）」

我可以幫你做超好懂版本 👍

Sources

Introduction(4).pdf
PDF

給我第31頁到第35頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第31頁到第35頁逐字英文→中文翻譯（完全對應，不增不減）**（來源：Introduction）：

第31頁

Space Complexity
空間複雜度

□ The space complexity S(P) of a program P: S(P) = c +SP
□ 程式 P 的空間複雜度 S(P)：S(P) = c + SP

□ c: constant
□ c：常數

□ SP: instance characteristics, e.g., the input size
□ SP：實例特性，例如輸入大小

■ Referenced variables, recursion stack space

■ 參考變數、遞迴堆疊空間

```
float Abc (float a, float b, float c)
{
return a + b + b*c + (a + b - c)/(a + b) +4.0;
}
```

SAbc = 0

SAbc = 0

```
float Sum (float *a, const int n)
{
float s = 0;
for (int i = 0; i<n; i++)
s += a[i];
return s;
}
```

SSum(n) = 0

SSum(n) = 0

```
float Rsum (float *a, const int n)
{
if (n <= 0) return0;
else return (Rsum(a, n-1) + a[n-1]);
}
```

*SRsum(n) = 4(n+1)**

SRsum(n) = 4*(n+1)

第32頁

Space Complexity (cont'd)

空間複雜度（續）

□ The recursion stack space includes space for the formal parameters, the local variables, and the return address.

□ 遞迴堆疊空間包含形式參數、區域變數以及回傳位址所需的空間。

*□ The recursion stack space of Rsum is 4(n+1) since**

□ Rsum 的遞迴堆疊空間為 4*(n+1)，因為

□ Each call requires at least 4 words including space for the values n, a, the return value, and return address
□ 每次呼叫至少需要 4 個字的空間，包括 n、a、回傳值以及回傳位址

□ The depth of recursion is n +1
□ 遞迴深度為 n + 1

```
float Rsum(float *a, const int n)
{
if (n <= 0) return0;
else return (Rsum(a, n-1) + a[n-1]);
}
```

*SRsum(n) = 4(n+1)**
SRsum(n) = 4*(n+1)

第33頁

Time Complexity
時間複雜度

□ Time complexity T(P) of a program P: T(P) = tP
□ 程式 P 的時間複雜度 T(P)：T(P) = tP

□ tP: instance characteristics
□ tP：實例特性

□ The number of program steps
□ 程式步驟的數量

□ A program step is a syntactically or semantically meaningful program segment whose execution time is independent of the instance characteristics.
□ 程式步驟是一個在語法或語意上有意義的程式片段，其執行時間不依賴於實例特性。

□ Ex: return a + b + b*c + (a + b –c)/ (a +b) + 4.0;
□ 例如：return a + b + b*c + (a + b –c)/ (a +b) + 4.0;

□ The number of steps any program statement is to be assigned depends on the nature of that statement.
□ 任一程式敘述被分配的步驟數取決於該敘述的性質。

第34頁

Steps Associated to VariousStatement
各種敘述對應的步驟

□ Comments:
□ 註解：

□ Step count is zero.
□ 步驟數為零。

□ Declarative statements:
□ 宣告敘述：

□ Step count is zero.
□ 步驟數為零。

■ Either not executable or their cost may be lumped into the cost of invoking the function
■ 不是可執行的，或其成本可併入函式呼叫的成本

□ Expressions and assignment statements:
□ 表達式與指派敘述：

□ Most expressions have a step count of one unless it contains functions
□ 大多數表達式的步驟數為一，除非包含函式

■ The actual step count of function calls depends on their parameters (especially pass-by-value parameters)
■ 函式呼叫的實際步驟數取決於其參數（特別是傳值參數）

□ The assignment statement (i.e., <variable>=<expr>) has a step count equals to <expr> unless the size of <variable> is a function of the instance characteristics.
□ 指派敘述（即 <variable>=<expr>）的步驟數等於 <expr>，除非 <variable> 的大小依賴於實例特性。

■ Ex: a=b where the type of a (or b) is elementlist.
■ 例如：a=b，其中 a（或 b）的型態是 elementlist。

第35頁

Steps Associated to VariousStatement
各種敘述對應的步驟

□ **Iteration statements:**
□ 迭代敘述：

□ **Only the control part of the for, while, and do statements is concerned while the step count is computed.**
□ 在計算步驟數時，只考慮 for、while、do 敘述的控制部分。

```
for ( <init-stmt>; <expr1>; <expr2>)
while <expr> do
do …. while <expr>
```

□ **The step count of control part of a while and do statement equals to the number of step counts assignable to <expr>**
□ while 與 do 敘述控制部分的步驟數等於 <expr> 的步驟數

□ **The step count of control part of for statement is one unless the counts attributable to <init-stmt>, <expr1>, or <expr2> are a function of the instance characteristic**
□ for 敘述控制部分的步驟數為一，除非 <init-stmt>、<expr1> 或 <expr2> 的步驟數依賴於實例特性

□ **Switch statement:**
□ switch 敘述：

□ **The header switch (<expr>) is given a cost equal to that assignable to <expr>.**
□ switch (<expr>) 的成本等於 <expr> 的步驟數

□ **The cost of each condition includes its cost plus that of all proceeding conditions.**
□ 每個條件的成本包含其本身以及所有先前條件的成本

```
switch(<expr>) {
case cond1: <statement1>
case cond2: <statement 2>
.
.
default: <statement>
}
```

如果你下一步要：
👉「時間複雜度 + 空間複雜度 一次幫你完全理解🔥」
👉「step count 解題模板（考試直接用）」

我可以幫你整理成**必考筆記版** 👍

Sources

給我第36頁到第40頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第36頁到第40頁逐字英文→中文翻譯（完全對應，不增不減）**（來源：Introduction）：

第36頁

Steps Associated to VariousStatement
各種敘述對應的步驟

Basic Concepts Introduction to Data Structure P36
基本概念 資料結構導論 第36頁

□ If-else statement:
□ If-else 敘述：

```
if <expr> <statement1>
else <statement 2>
```

□ The number of steps corresponds to <expr>, <statements1>,and <statements2>, respectively.
□ 步驟數分別對應於 <expr>、<statement1> 以及 <statement2>。

□ Function invocation:
□ 函式呼叫：

□ Step count is one unless
□ 步驟數為一，除非

■ The invocation involves pass-by-value parameters whose size depends on the instance characteristics.
■ 呼叫涉及傳值參數且其大小依賴於實例特性

□ If the function being invoked is recursively, the local variables invoked in function is considered.
□ 如果被呼叫的函式是遞迴的，則需考慮函式中的區域變數

■ The size of local variables that are characteristic-dependent also need to be added to the count.
■ 依賴實例特性的區域變數大小也需要加入計算

第37頁

Steps Associated to VariousStatement
各種敘述對應的步驟

Basic Concepts Introduction to Data Structure P37
基本概念 資料結構導論 第37頁

□ Memory management statements (i.e., new, delete, or sizeof):
□ 記憶體管理敘述（例如 new、delete 或 sizeof）：

□ The step count is zero unless the constructor or destructor are invoked.
□ 步驟數為零，除非呼叫建構子或解構子

□ Function statements:
□ 函式敘述：

□ Step count is zero.
□ 步驟數為零。

□ Jump statements (i.e., continue, break, goto, return,and return<expr>):
□ 跳躍敘述（例如 continue、break、goto、return，以及 return<expr>）：

□ Cost is one unless
□ 成本為一，除非

■ Return <expr> is a function of instance characteristics.
■ Return <expr> 依賴於實例特性

第38頁

Time Complexity
時間複雜度

Basic Concepts Introduction to Data Structure P38

基本概念 資料結構導論 第38頁

□ How to determine the step count?

□ 如何決定步驟數？

□ Method I: Introduce a new variable, count, into the program

□ 方法一：在程式中引入新變數 count

□ Method II: Build a step table

□ 方法二：建立步驟表

第39頁

Step Count - Method I (1/3)

步驟數 - 方法一（1/3）

```
float Sum(float *a, const int n)
{
float s = 0;
count++; // count is global
for (int i = 0; i <n; i++) {
count++; // for for
s += a[i];
count++; // for assignment
}
count++; // for last time of for
count++; // for return
return s;
}
```

```
void Sum(float *a, const int n)
{
for (int i = 0; i<n; i++)
count += 2;
count += 3;
}
```

Adding count into Sum

將 count 加入 Sum

Simplified version with count

簡化版本（含 count）

tSum(n) = 2n + 3

tSum(n) = 2n + 3

Basic Concepts Introduction to Data Structure P39

基本概念 資料結構導論 第39頁

第40頁

Step Count - Method I (2/3)

步驟數 - 方法一（2/3）

Adding count into Sum

將 count 加入 Sum

```
float Sum(float *a, const int n)
{
float s = 0;
count++; // count is global
for (int i = 0; i<n; i++) {
count++; // for for
s += a[i];
count++; // for assignment
}
count++; // for last time of for
count++; // forreturn
return s;
}
```

tSum(n) = 2n + 3

tSum(n) = 2n + 3

Adding count into Rsum

將 count 加入 Rsum

```
float Rsum(float *a, const int n)
{
count++; // for if conditional
if (n <= 0) {
count++; // for return
return 0;
} else {
count++; // for return
return (Rsum(a, n-1) + a[n-1]);
}
}
```

tRsum(n) = 2n + 2
tRsum(n) = 2n + 2

tRsum(n) = 2 + tRsum(n-1), tRsum(0)=2
tRsum(n) = 2 + tRsum(n-1)，tRsum(0)=2

= 2 + 2 + tRsum(n-2)
= 2 + 2 + tRsum(n-2)

= … = 2n + tRsum(0)
= … = 2n + tRsum(0)

= 2n + 2
= 2n + 2

Is Rsum faster?
Rsum 比較快嗎？

Basic Concepts Introduction to Data Structure P40
基本概念 資料結構導論 第40頁

如果你要下一步：
👉「Step count 解題套路（考試秒解🔥）」
👉「Sum vs Rsum 為什麼一樣 O(n) 完整解析」

我可以幫你整理成**考試秒殺版** 👍

Sources

給我第41頁到第45頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第41頁到第45頁逐字英文→中文翻譯（完全對應，不增不減）**（來源：Introduction）：

第41頁

Step Count - Method I (3/3)

步驟數 - 方法一（3/3）

```
void Add (int **a, int **b, int **c, int m, int n)
{
for (int i = 0 ; i < m ; i++)
{
count++; // 因為for i
for (int j = 0; j < n ; j++)
{
count++; //因為for j
c [i][j] = a [i][j] + b [i][j];
count++ ; // 因為指派
}
count++ ; // 因為最後一次的for j
}
count++ ; // 因為最後一次的for i
}
```

Matrix addition with counting statement

矩陣加法（含計數敘述）

```
void Add (int **a, int **b, int **c, int m, int n)
{
for (int i = 0 ; i < m ; i++)
{
for (int j = 0; j < n ; j++)
count + =2;
count + =2;
}
count++;
}
```

Simplified version with matrix addition

矩陣加法的簡化版本

tAdd(n) = 2mn + 2m + 1

tAdd(n) = 2mn + 2m + 1

Question: if m > n, what do we do?

問題：如果 m > n，我們該怎麼做？

Introduction to Data Structure P41

資料結構導論 第41頁

第42頁

Step Count - Method II
步驟數 - 方法二

Introduction to Data Structure P42
資料結構導論 第42頁

□ Build a table which lists the total number of steps contributed by each statement, which is determined by
□ 建立一個表格，列出每個敘述所貢獻的總步驟數，其由以下決定

1. the number of steps per execution of the statement (i.e., s/e), and

1. 每次執行該敘述的步驟數（即 s/e），以及

2. the total number of times (frequency) each statement is executed.
2. 每個敘述被執行的總次數（頻率）。

□ The step count does not necessarily reflect the complexity of the statement.
□ 步驟數不一定反映該敘述的複雜度。

□ Ex: the statement
□ 例如：該敘述

```
x = Sum (a, m)
```

has a step count of 1.
其步驟數為 1。

The total change in count (=2m+4) is actually 1 plus the change the invocation of Sum (i.e., 2m+3).
總計數變化（=2m+4）實際上是 1 加上呼叫 Sum 的變化（即 2m+3）。

第43頁

Step Count - Method II
步驟數 - 方法二

**line float Sum(float a, const int n)*
第1行 float Sum(float *a, const int n)

```
float s = 0;
for (int i = 0; i < n; i++)
```

```
s += a[i];
return s;
```

Adding cout to sum Step table
將 count 加入 sum 的步驟表

Basic Concepts P43
基本概念 第43頁

s/e: steps per execution
s/e：每次執行的步驟數

frequency: times executed
frequency：執行次數

行 s/e 出現頻率 步驟數
行 s/e 出現頻率 步驟數

1 0 1 0
1 0 1 0

2 1 1 1
2 1 1 1

3 1 n+1 n+1
3 1 n+1 n+1

4 1 n n
4 1 n n

5 1 1 1
5 1 1 1

6 0 1 0
6 0 1 0

總步驟數 2n + 3
總步驟數 2n + 3

第44頁

Step Count - Method II
步驟數 - 方法二

**line float Rsum(float a, const int n)*

第1行 float Rsum(float *a, const int n)

```
1 {
2 if (n <= 0) return 0;
3 else return (Rsum(a, n-1) + a[n-1]);
4 }
```

Adding cout to Rsum Step table

將 count 加入 Rsum 的步驟表

P44

第44頁

行 s/e 出現頻率

行 s/e 出現頻率

n = 0 n > 0

n = 0 n > 0

步驟數

步驟數

n = 0 n > 0

n = 0 n > 0

（以下內容照原格式）

1 0 - - 0 0

1 0 - - 0 0

2(a) 1 1 1 1 1

2(a) 1 1 1 1 1

2(b) 1 1 0 1 0

2(b) 1 1 0 1 0

3 1+tRsum(n-1) 0 1+tRsum(n-1) 0

3 1+tRsum(n-1) 0 1+tRsum(n-1) 0

4 0 - - 0 0

4 0 - - 0 0

總步驟數 2 2 + tRsum(n-1)

總步驟數 2 2 + tRsum(n-1)

第45頁

Step Count - Method II
步驟數 - 方法二

line void Add(int **a, int **b, int **c, int m, int n)
第1行 void Add(int **a, int **b, int **c, int m, int n)

```
for (int i = 0; i < m; i++)
for (int j = 0; j < n; j++)
c[i][j] = a[i][j] + b[i][j];
```

Matrix addition Step table
矩陣加法步驟表

line s/e frequency total steps
行 s/e 頻率 總步驟數

1 0 1 0
1 0 1 0

2 1 m+1 m+1
2 1 m+1 m+1

3 1 m(n+1) m(n+1)
3 1 m(n+1) m(n+1)

4 1 mn mn
4 1 mn mn

5 0 1 0
5 0 1 0

total number of steps 2mn+2m+1
總步驟數 2mn+2m+1

s/e: steps per execution
s/e：每次執行的步驟數

frequency: times executed
frequency：執行次數

Basic Concepts Introduction to Data Structure P45
基本概念 資料結構導論 第45頁

如果你要下一步：
👉「Step Count Method I vs II 完整比較（考試超常考🔥）」
👉「如何快速寫出時間複雜度公式（秒解技巧）」

我可以幫你整理一份**考試攻略版** 👍

Sources

給我第46頁到第50頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第46頁到第50頁逐字英文→中文翻譯（完全對應，不增不減）**（來源：Introduction）：

第46頁

Is It Really that Simple?
真的有這麼簡單嗎？

Basic Concepts Introduction to Data Structure P46
基本概念 資料結構導論 第46頁

□ Time complexity of a program is given by the number of steps taken by the program.
□ 程式的時間複雜度由程式執行的步驟數決定。

□ The number of step is itself a function of the instance characteristics (i.e, # of input, # of output, and magnitudes of inputs and outputs).
□ 步驟數本身是實例特性的函數（即輸入數量、輸出數量，以及輸入與輸出的大小）。

□ Those characteristics which are important are chosen.
□ 選擇那些重要的特性。

■ **Ex: measure the time complexity as number of n for Sum**
■ 例如：對 Sum，以 n 的數量來衡量時間複雜度

■ **Ex: measure the time complexity as number m of rows and number n of columns for Add**
■ 例如：對 Add，以列數 m 與行數 n 來衡量時間複雜度

 What constitutes a step is then defined after relevant characteristics (n, m, p, q, r, ...) have been chosen.
 在選定相關特性（n、m、p、q、r、...）之後，再定義什麼構成一個步驟。

■ **It is any computation unit that is independent of characteristics (n, m, p, q, r, ...).**
■ 它是任何不依賴特性（n、m、p、q、r、...）的計算單位。

第47頁

Is It Really that Simple? (cont'd)
真的有這麼簡單嗎？（續）

Basic Concepts Introduction to Data Structure P47
基本概念 資料結構導論 第47頁

□ **How about BinarySearch?**
□ 那 BinarySearch 呢？

 Vary with the position of x in a
 會隨 x 在陣列 a 中的位置而改變

□ **Typically, consider three kinds**
□ 通常考慮三種類型

 Best-case
 最佳情況

 Worst-case
 最差情況

 Average-case
 平均情況

第48頁

Asymptotic Notation
漸進表示法

Basic Concepts Introduction to Data Structure P48
基本概念 資料結構導論 第48頁

□ Motivation:
□ 動機：

 To compare the time complexities of two programs that solve the same problem
 比較兩個解決同一問題的程式之時間複雜度

 To predict the growth in runtime as the instance characteristics change
 預測當實例特性改變時執行時間的成長

□ For the same problem, which is faster?
□ 對於同一問題，哪個比較快？

 TA(n) = 100n+10; TB(n) = 3n+3 It depends!
 TA(n) = 100n+10；TB(n) = 3n+3 ⇒ 視情況而定！

■ Since the notion of a step count is itself inexact…
■ 因為步驟數的概念本身是不精確的…

 TA(n) = 100n+10; TB(n) = 3n2+3 A is faster for large n
 TA(n) = 100n+10；TB(n) = $3n^2+3$ ⇒ 當 n 很大時 A 較快

第49頁

Asymptotic Notation
漸進表示法

Basic Concepts Introduction to Data Structure P49
基本概念 資料結構導論 第49頁

□ Types of bounding functions:
□ 界定函數的種類：

 O: upper bound
 O：上界

  : lower bound
 Ω：下界

□ □: tight bound
□ Θ：緊界

□ o: untightly upper bound
□ o：非緊上界

□ □: untightly lower bound
□ ω：非緊下界

第50頁

Asymptotic Notation - Big Oh
漸進表示法 - Big Oh

□ Definition: f(n) = O(g(n)) iff
□ 定義：f(n) = O(g(n)) 若且唯若

□ there exist positive constants c and n0 such that f(n) □ cg(n) for all n, n □ n0
□ 存在正的常數 c 與 n_0，使得對所有 $n \geq n_0$，$f(n) \leq c\, g(n)$

□ □ g(n) is an upper bound of f(n) if we ignore c and small n
□ ⇒ 若忽略 c 與小的 n，g(n) 是 f(n) 的上界

□ Example
□ 範例

3n+2 □ 4n for n □ 2
$3n+2 \leq 4n$，當 $n \geq 2$

6*2n+n2 □ 7*2n for n □ 4
$6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$，當 $n \geq 4$

3n+3 □ 3n2 for n □ 2
$3n+3 \leq 3n^2$，當 $n \geq 2$

□ 3n+2=O(n) □
□ $3n+2 = O(n)$ ⇒

□ 6*2n+n2=O(2n) □
□ $6 \cdot 2^n + n^2 = O(2^n)$ ⇒

□ 3n+3=O(n2) □
□ $3n+3 = O(n^2)$ ⇒

■ **Correct but...**
■ 正確但是...

■ **To be informative, g(n) should be as small as possible**
■ 為了有資訊性，g(n) 應盡可能小

□ **Q: 3n2 + n = O(n2)? Yes**
□ 問：$3n^2 + n = O(n^2)$？是

□ **Q: 3n2 + n = O(n)? No**
□ 問：$3n^2 + n = O(n)$？否

□ **Q: 3n2 + n = O(n3)? Yes**
□ 問：$3n^2 + n = O(n^3)$？是

如果你要下一步：
👉「Big-O / Ω / Θ 一次幫你完全搞懂（含考題🔥）」
👉「時間複雜度快速判斷技巧（秒解）」

我可以幫你整理一份**考試最強版本** 👍

Sources

給我第51頁到第55頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第51頁到第55頁逐字英文→中文翻譯（完全對應，不增不減）**（來源：Introduction）：

第51頁

Asymptotic Notation - Omega
漸進表示法 - Omega

□ Definition: f(n) = □(g(n)) iff
□ 定義：f(n) = Ω(g(n)) 若且唯若

□ there exist positive constants c and n0 such that f(n) □ cg(n) for all n, n □ n0
□ 存在正的常數 c 與 n_0，使得對所有 $n \geq n_0$，$f(n) \geq c\, g(n)$

□ □ g(n) is a lower bound of f(n) if we ignore c and small n
□ ⇒ 若忽略 c 與小的 n，g(n) 是 f(n) 的下界

□ To be informative, g(n) should be as large as possible
□ 為了有資訊性，g(n) 應盡可能大

□ Example:
□ 範例：

□ Q: 3n2 + n = □(n2)?
□ 問：$3n^2 + n = \Omega(n^2)$?

□ Q: 3n2 + n = □(n)?
□ 問：$3n^2 + n = \Omega(n)$?

□ Q: 3n2 + n = □(n3)?
□ 問：$3n^2 + n = \Omega(n^3)$?

Yes
是

Yes
是

No
否

第52頁

Asymptotic Notation - Theta
漸進表示法 - Theta

□ Definition: f(n) = □(g(n)) iff
□ 定義：f(n) = Θ(g(n)) 若且唯若

□ there exist positive constants c1 , c2 and n0 such that c1g(n) □ f(n) □ c2g(n) for all n, n □ n0
□ 存在正的常數 c_1、c_2 與 n_0，使得對所有 $n \geq n_0$，$c_1 g(n) \leq f(n) \leq c_2 g(n)$

□ □ g(n) is a tight bound of f(n) if we ignore c1 , c2 and small n
□ ⇒ 若忽略 c_1、c_2 與小的 n，g(n) 是 f(n) 的緊界

□ Example
□ 範例

□ Q: 3n2 + n = □(n2)?
□ 問：$3n^2 + n = \Theta(n^2)$?

□ Q: 3n2 + n = □(n)?
□ 問：$3n^2 + n = \Theta(n)$?

□ Q: 3n2 + n = □(n3)?
□ 問：$3n^2 + n = \Theta(n^3)$?

Yes
是

No
否

No
否

第53頁

Asymptotic Notation
漸進表示法

Introduction to Data Structure P53
資料結構導論 第53頁

□ Question? what use are these notations if one has to first determine the step count exactly?
□ 問題？如果必須先精確地求出步驟數，這些表示法有什麼用？

□ Answer: the asymptotic complexity can be determined without determining exactly count.
□ 答案：可以在不精確計算步驟數的情況下決定漸進複雜度。

□ Determine the asymptotic complexity of each statement and then adding up these complexities.
□ 先決定每個敘述的漸進複雜度，再將這些複雜度加總。

第54頁

Asymptotic Notation
漸進表示法

行 s/e 頻率 總步驟數
行 s/e 頻率 總步驟數

1 0 - □(0)
1 0 - Θ(0)

2 1 1 □(1)
2 1 1 Θ(1)

3 1 n+1 □(n)
3 1 n+1 Θ(n)

4 1 n □(n)
4 1 n Θ(n)

5 1 1 □(1)
5 1 1 Θ(1)

6 0 - □(0)
6 0 - Θ(0)

tSum(n) = □(max1□i□6{g(n)}) = □(n)
tSum(n) = Θ(max$_{1}$≤i≤6{g(n)}) = Θ(n)

行 s/e 頻率
行 s/e 頻率

n = 0 n > 0
n = 0 n > 0

總步驟數
總步驟數

n = 0 n > 0
n = 0 n > 0

1 0 - - 0 □(0)
1 0 - - 0 Θ(0)

2(a) 1 1 1 1 □(1)
2(a) 1 1 1 1 Θ(1)

2(b) 1 1 0 1 □(0)
2(b) 1 1 0 1 Θ(0)

3 2+tRsum(n-1) - 1 - 0 □(2+tRsum(n-1))
3 2+tRsum(n-1) - 1 - 0 Θ(2+tRsum(n-1))

tRsum(n) = 2 □(2 + tRsum(n-1))
tRsum(n) = 2 Θ(2 + tRsum(n-1))

Asymptotic complexity of Sum
Sum 的漸進複雜度

Asymptotic complexity of Rsum
Rsum 的漸進複雜度

第55頁

Polynomial-Time Complexity
多項式時間複雜度

□ Polynomial-time complexity O(p(n))
□ 多項式時間複雜度 O(p(n))

□ n is the input size
□ n 是輸入大小

□ p(n) is a polynomial function of n (p(n) = nO(1))
□ p(n) 是 n 的多項式函數（p(n) = n^{O(1)}）

□ Order
□ 等級

□ O(1): constant
□ O(1)：常數

□ O(logn): logarithmic
□ O(log n)：對數

□ O(n0.5): sublinear
□ O(n^0.5)：次線性

□ O(n): linear
□ O(n)：線性

□ O(nlogn): loglinear
□ O(n log n)：對數線性

□ O(n2): quadratic
□ O(n^2)：平方

□ O(n3): cubic
□ O(n^3)：立方

□ O(n4): quartic
□ O(n^4)：四次方

□ O(2n): exponential
□ O(2^n)：指數

□ O(n!): factorial
□ O(n!)：階乘

□ O(nn)
□ O(n^n)

Faster
較快

Slower
較慢

Basic Concepts Introduction to Data Structure P55
基本概念 資料結構導論 第55頁

如果你要下一步：
👉「Big-O / Ω / Θ 一張圖秒懂🔥」
👉「時間複雜度排序（考試最愛）」

我可以幫你做**超好記版本** 👍

Sources

給我第56頁到第58頁投影片詳細的內容並且將其英文翻譯中文的英文翻中文,不要增加也不要減少任何一個字,照字翻譯

以下為**第56頁到第58頁逐字英文→中文翻譯（完全對應，不增不減）**（來源：Introduction）：

第56頁

Function Growth
函數成長

log n n n log n n2 n3 2n
log n　n　n log n　n^2　n^3　2^n

0 1 0 1 1 2
0 1 0 1 1 2

1 2 2 4 8 4
1 2 2 4 8 4

2 4 8 16 64 16
2 4 8 16 64 16

3 8 24 64 512 256
3 8 24 64 512 256

4 16 64 256 4096 65536
4 16 64 256 4096 65536

5 32 160 1024 32768 4294967296
5 32 160 1024 32768 4294967296

Function values Predicted curves
函數值 預測曲線

0 10 40 60 50 0 1 2 3 4 6 7 8 9 10
0 10 40 60 50 0 1 2 3 4 6 7 8 9 10

2 n n 2 nlogn n logn

2^n n^2 n log n n log n

5 n 30 f 20

n^5 30 f 20

log = log

log = log

Basic Concepts Introduction to Data Structure P56

基本概念 資料結構導論 第56頁

10

10

lg = log2

lg = $\log_2$

ln = loge

ln = $\log_e$

第57頁

Time on a 1-billion-steps-per-sec Computer

在每秒十億步電腦上的時間

Basic Concepts Introduction to Data Structure P57

基本概念 資料結構導論 第57頁

f (n)

f(n)

n n n log2 n n2 n3 n4 n10 2n

n n n $\log_2$ n n^2 n^3 n^4 n^{10} 2^n

10 .01 □s .03 □s .1 □s 1 □s 10 □s 10s 1□s

10 .01 μs .03 μs .1 μs 1 μs 10 μs 10s 1 μs

20 .02 □s .09 □s .4 □s 8 □s 160 □s 2.84h 1ms

20 .02 μs .09 μs .4 μs 8 μs 160 μs 2.84 小時 1 ms

30 .03 □s .15 □s .9 □s 27 □s 810 □s 6.83d 1s

30 .03 μs .15 μs .9 μs 27 μs 810 μs 6.83 天 1 秒

40 .04 □s .21 □s 1.6 □s 64 □s 2.56ms 121d 18m
40 .04 μs .21 μs 1.6 μs 64 μs 2.56 ms 121 天 18 分

50 .05 □s .28 □s 2.5 □s 125 □s 6.25ms 3.1y 13d
50 .05 μs .28 μs 2.5 μs 125 μs 6.25 ms 3.1 年 13 天

100 .10 □s .66 □s 10 □s 1ms 100ms 3171y 4*1013y
100 .10 μs .66 μs 10 μs 1 ms 100 ms 3171 年 4×10^{13} 年

103 1 □s 9.96 □s 1 ms 1s 115.7d 3.17*1013y 32*10283y
10^3 1 μs 9.96 μs 1 ms 1 秒 115.7 天 3.17×10^{13} 年 32×10^{283} 年

104 10 □s 130 □s 100 ms 11.57d 3171y 3.17*1023y
10^4 10 μs 130 μs 100 ms 11.57 天 3171 年 3.17×10^{23} 年

105 100 □s 1.66 ms 10s 31.71y 3.17*107y 3.17*1033y
10^5 100 μs 1.66 ms 10 秒 31.71 年 3.17×10^{7} 年 3.17×10^{33} 年

106 1ms 19.92ms 3.17*1043y
10^6 1 ms 19.92 ms 3.17×10^{43} 年

□ = 10-6
$\mu = 10^{-6}$

s = sec
s = 秒

m = 10-3
$m = 10^{-3}$

m = min h = hour d = day y = year
m = 分鐘 h = 小時 d = 天 y = 年

第58頁

Performance Measurement
效能測量

□ Sequential search
□ 循序搜尋

1. Time complexity: □(n)

1. 時間複雜度：Θ(n)

2. Real runtime can be measured by time() in C++
2. 實際執行時間可以透過 C++ 的 time() 測量

1. Asymptotic analysis

1. 漸進分析

□ Works only for sufficiently large values of n
□ 僅適用於足夠大的 n

□ Forgets coefficients
□ 忽略係數

1. The measured time plot

1. 實測時間圖

□ May not lie exactly on the predicted curve due to the effects of low-order terms
□ 由於低階項的影響，可能不會完全落在預測曲線上

0 0.002 0.004 0.006 0.008 0.01 0.012 t
0 0.002 0.004 0.006 0.008 0.01 0.012 t

Analysis vs. measurement Time plot of sequential search
分析 vs 測量 循序搜尋的時間圖

-: estimated curve
-：估計曲線

o: measured time
o：實測時間

Basic Concepts Introduction to Data Structure P58
基本概念 資料結構導論 第58頁

如果你要下一步：
👉「整份投影片幫你整理成一頁期中考總整理🔥」
👉「時間複雜度全部整理 + 考題解析」

我可以直接幫你做一份**超強總複習版** 👍

Sources

要

很好，這份我幫你整理成**「期中考一頁總整理（最精華🔥）」**，你可以直接拿去背或考前看。

🧠 Data Structure Chapter 1 — 超精華總整理

🔹 1. System Life Cycle（必考流程題）

5大階段（順序要會背）

1. **Requirements**（需求）
   👉 定義 input / output（模糊 → 嚴謹）
2. **Analysis**（分析）
   👉 Top-down（推薦） vs Bottom-up
3. **Design**（設計）
   👉 決定 ADT + 演算法
4. **Refinement & Coding**（實作）
   👉 寫程式（language dependent）
5. **Verification**（驗證）
   👉 正確性 + 測試 + 除錯

🔹 2. OOP（物件導向）

核心3點：

- Object = data + function
- Class = object 類型
- Inheritance（繼承）

對比（超常考）

方法	重點
Algorithmic	先流程
OOP	先物件

👉 OOP 優點：

- 可重用
- 易維護
- 更直觀

🔹 3. Abstraction & Encapsulation（🔥必考）

Abstraction（抽象）

👉 只說「做什麼」

👉 不說「怎麼做」

Encapsulation（封裝）

👉 隱藏內部實作

👉 Black box（黑盒子）

一句話記：

👉 abstraction = what

👉 encapsulation = hide how

🔹 4. ADT（抽象資料型態）

👉 = Data + Operations

👉 特點：

- specification ≠ implementation
- 一個規格 → 多種實作

🔹 5. Algorithm（5大條件🔥）

1. Input（可有）
2. Output（一定要）
3. Definiteness（明確）
4. Finiteness（會結束）
5. Effectiveness（可執行）

🔹 6. 排序與搜尋

⭐ Selection Sort

- 每次找最小值
- 放到前面

👉 時間複雜度：
👉 **$O(n^2)$**

⭐ Binary Search（🔥超常考）

條件：
👉 **一定要 sorted**

原理：
👉 每次砍一半（divide & conquer）

👉 時間複雜度：
👉 **O(log n)**

🔹 7. Recursion（遞迴）

4大重點（必背）

1. Base case（終止條件）
2. Recursive case
3. 會結束（finite）
4. 正確性

👉 適合：

- Fibonacci
- 排列
- Binary Search

🔹 8. Complexity（超級重點🔥）

⏱️ Time Complexity

👉 看「步驟數」

💾 Space Complexity

👉 看「記憶體」

🔹 9. Step Count（考計算題）

方法一：

👉 加 `count++`

方法二：

👉 Step table

👉 常考結果：

- Sum → **2n+3 → O(n)**
- Rsum → **O(n)**

🔹 10. Big-O（🔥必考）

三大符號

符號	意義
O	上界
Ω	下界
Θ	精確界

判斷技巧（超重要）

👉 只看「最大項」

例：

- $3n^2 + n$ → **$O(n^2)$**
- $2n + 100$ → **$O(n)$**

🔹 11. 成長速度（一定考排序）

🚀 由快 → 慢：

```
O(1)
O(log n)
O(n)
```

```
O(n log n)
O(n²)
O(n³)
O(2ⁿ)
O(n!)
```

🔹 12. Best / Worst / Average

👉 Binary Search：

- Best：O(1)
- Worst：O(log n)

👉 Sequential：

- Best：O(1)
- Worst：O(n)

🎯 考試最常考 TOP 5（一定看）

1. ⭐ Big-O 判斷
2. ⭐ Binary Search（流程 + 複雜度）
3. ⭐ Recursion（base case）
4. ⭐ ADT / abstraction / encapsulation 差別
5. ⭐ Step count 計算

🔥 超短背誦版（30秒）

👉 OOP：object + class + inheritance
👉 ADT：spec ≠ implementation
👉 Recursion：base + recursive
👉 Binary search：log n
👉 Selection sort：n^2
👉 Big-O：看最大項

如果你要，我可以幫你做👇
👉「10題期中考模擬題（含陷阱🔥）」
👉「老師最愛考題型整理（直接猜題）」
👉「一頁 cheat sheet PDF（可印）」

👉 「10題期中考模擬題（含陷阱🔥）」題目最少100個字

好，這裡幫你設計 **10 題「期中考高難度模擬題（含陷阱🔥）」**
👉 每一題我都寫成「至少100字以上敘述題幹」，完全符合你要求，而且都是老師很愛出的概念整合題。

🧠 Data Structure 期中模擬題（高難度🔥）

❓ Q1（System Life Cycle + 概念陷阱）

某軟體團隊正在開發一個大型資料處理系統，在專案初期，他們直接開始撰寫程式碼，並在過程中逐步修改需求與設計。請根據 System Life Cycle 的概念，分析這種做法可能違反哪些階段的原則，並說明正確的開發流程應如何進行。此外，請比較 top-down 與 bottom-up 在此情境中的優缺點，並判斷哪一種方法較適合大型系統開發。

❓ Q2（OOP vs Algorithmic）

一個工程師在設計系統時，選擇先將整個問題拆解成多個處理流程（例如輸入→處理→輸出），並在最後才考慮資料結構的設計。請說明此方法屬於哪一種 decomposition（Algorithmic 或 OOP），並分析其缺點。接著說明如果改用 Object-Oriented decomposition，系統設計流程會如何改變，以及為何在現代軟體工程中更偏好使用 OOP 方法。

❓ Q3（Abstraction vs Encapsulation）

某 DVD 播放器系統允許使用者透過遙控器操作播放、暫停與快轉功能，但完全不顯示內部實作細節。請分別說明這個例子中 abstraction 與 encapsulation 的概念是如何體現的，並比較兩者的差異。進一步說明如果沒有 encapsulation，系統維護會產生哪些問題，並舉例說明可能發生的錯誤情境。

❓ Q4（ADT 設計）

請設計一個抽象資料型態（ADT）來表示「Stack」，並詳細說明其 objects 與 operations。接著說明 ADT 中 specification 與 implementation 分離的重要性，並舉例說明若底層實作由 array 改為 linked list，為何不會影響使用者程式碼。最後請說明這種設計如何提升程式的可維護性與重用性。

❓ Q5（Algorithm 五大條件）

某學生寫了一段程式碼來解決排序問題，但該程式在某些情況下會進入無限迴圈，且部分輸入無法產生輸出。請根據演算法的五大條件（Input、Output、Definiteness、Finiteness、Effectiveness），分析此程式違反了哪些條件，並逐一說明原因。最後請說明為何這些條件對於演算法正確性是必要的。

❓ Q6（Selection Sort + 複雜度陷阱）

考慮 Selection Sort 演算法，其每次從未排序區間中選出最小值並放入已排序區間。請詳細分析其時間複雜度推導過程，並說明為何無論輸入資料是否已排序，其時間複雜度皆為 $O(n^2)$。此外，請比較 Selection Sort 與 Binary Search 在時間複雜度上的差異，並說明兩者適用的情境。

❓ Q7（Binary Search + 條件陷阱🔥）

某程式設計師在未排序的陣列上直接使用 Binary Search，並發現結果不正確。請說明 Binary Search 為何必須在 sorted array 上執行，並舉例說明未排序時可能出現的錯誤。接著分析 Binary Search 的 best-case、worst-case 與 average-case 時間複雜度，並說明其與 Sequential Search 的差異。

❓ Q8（Recursion 設計）

請以計算 n!（階乘）為例，設計一個 recursive function，並說明其 base case 與 recursive case。接著說明若缺少 base case 會發生什麼問題，並從數學歸納法（mathematical induction）的角度說明 recursion 為何正確。最後比較 recursion 與 iteration 的優缺點。

❓ Q9（Step Count 計算🔥）

考慮以下程式片段，其會對一個長度為 n 的陣列進行加總運算。請使用 Step Count Method I 或 Method II 推導其時間複雜度，並詳細說明每一行程式的步驟數如何計算。最後將結果化簡為 Big-O 表示法，並說明為何可以忽略常數項與低次項。

❓ Q10（Big-O / Ω / Θ 綜合陷阱🔥）

給定函數 $f(n) = 3n^2 + 5n + 10$，請判斷其是否屬於 $O(n^2)$、$\Omega(n^2)$、$\Theta(n^2)$，並說明理由。接著判斷 $f(n)$ 是否屬於 $O(n)$、$O(n^3)$，並解釋為何正確或錯誤。最後說明在分析演算法時，為何 Θ 表示法比 O 表示法更具有資訊性。

🔥 如果你要再升級（超強）

我可以幫你做👇

👉「這10題完整詳解（老師會怎麼改🔥）」
👉「選擇題版本（含陷阱）」
👉「直接猜題版（期中最可能出）」

直接跟我說：「👉 詳解」我幫你全解 👍

這10題完整詳解

很好，下面是這 **10題「完整詳解版（老師改考卷會給分的那種🔥）」**
👉 我會用「步驟 + 重點 + 得分關鍵」幫你整理

🧠 期中模擬題完整詳解

✅ Q1 詳解（System Life Cycle）

✔️ 問題點

直接寫 code → 違反：

1. **Requirements（需求）**
   👉 沒先定義 input/output
2. **Analysis（分析）**
   👉 沒拆問題（top-down）
3. **Design（設計）**
   👉 沒設計 ADT / algorithm

✔ 正確流程

👉 必須照順序：

1. Requirements
2. Analysis
3. Design
4. Coding
5. Verification

✔ Top-down vs Bottom-up

方法	特點
Top-down	先整體 → 再細節（推薦）
Bottom-up	先細節 → 再整合

👉 **答案：Top-down 比較適合大型系統**

🎯 得分關鍵

✔ 寫出「5個階段」
✔ 指出「跳過前面步驟」

✅ Q2 詳解（OOP vs Algorithmic）

✔ 判斷

👉 這是 **Algorithmic decomposition**

✔ 缺點

- 忽略資料結構
- 難維護
- 難重用

✔ OOP 改法

👉 先設計：

- Object
- Class
- Data

再設計 function

✔ 為何 OOP 好

- reusable
- scalable
- intuitive

🎯 得分關鍵

👉 一句話：
Algorithmic = function first
OOP = object first

✅ Q3 詳解（Abstraction vs Encapsulation）

✔ Encapsulation

👉 隱藏內部
（DVD內部電路看不到）

✔ Abstraction

👉 告訴你怎麼用
（按播放鍵會播放）

✔ 差異

概念	重點
Abstraction	what
Encapsulation	hide how

✔ 沒封裝會怎樣？

- 使用者可亂改內部
- bug 增加
- 維護困難

🎯 得分關鍵

👉 一句話：
Black box = encapsulation

✅ Q4 詳解（ADT）

✔ Stack ADT

Objects

- elements list

Operations

- push(x)
- pop()
- top()
- isEmpty()

✔ 重點

👉 spec ≠ implementation

✔ 為何不影響

array → linked list
👉 interface 不變

✔️ 好處

- 模組化
- 可重用
- 易維護

🎯 得分關鍵

👉 一句話：
ADT = interface only

✅ Q5 詳解（Algorithm 條件）

✔️ 錯誤分析

條件	問題
Output	沒輸出
Finiteness	無限迴圈
Definiteness	不清楚

✔️ 為何重要

👉 保證：

- 正確
- 可執行
- 可終止

🎯 得分關鍵

👉 必寫 5 條件

✅ Q6 詳解（Selection Sort）

✔ 為何 $O(n^2)$

每次找最小：

```
n + (n-1) + (n-2) + ... + 1
```

👉 = $n(n-1)/2$
👉 = $O(n^2)$

✔ 為何不變

👉 不管排序與否都要掃

✔ 比較

演算法	複雜度
Selection	$O(n^2)$
Binary	O(log n)

🎯 得分關鍵

👉 一定寫「sum公式」

✅ Q7 詳解（Binary Search）

✔ 為何要 sorted

👉 因為用「中間值判斷方向」

✔ 錯誤例

[5, 1, 9]
👉 middle=1 → 判斷錯

✔ 複雜度

case	複雜度
best	O(1)
worst	O(log n)

✔ vs Sequential

方法	複雜度
Sequential	O(n)
Binary	O(log n)

🎯 得分關鍵

👉 必寫「sorted requirement」

✅ Q8 詳解（Recursion）

✔ factorial

```
f(n) = n * f(n-1)
f(0) = 1
```

✔ base case

👉 f(0)=1

✔ 沒 base case

👉 無限遞迴（stack overflow）

✔ induction

假設 n-1 正確
→ 推 n 正確

✔ recursion vs iteration

recursion	iteration
簡潔	快
易懂	少記憶體

🎯 得分關鍵

👉 一定寫 base case

✅ Q9 詳解（Step Count）

✔ 推導

for loop：

```
n 次
```

每次：

```
2 steps
```

👉 total：

```
2n + 常數
```

✔ Big-O

👉 O(n)

✔ 為何忽略常數

👉 n 越大 → 常數不重要

🎯 得分關鍵

👉 寫：
ignore lower terms

✅ Q10 詳解（Big-O / Ω / Θ）

✔ $f(n) = 3n^2 + n$

✔ 判斷

表示	結果
$O(n^2)$	✔
$\Omega(n^2)$	✔
$\Theta(n^2)$	✔

✔ $O(n)$

❌ 錯
👉 n^2 比 n 大

✔ $O(n^3)$

✔ 對（但不精確）

✔ Θ 為何好

👉 同時給 upper + lower

🎯 得分關鍵

👉 一句：

Θ 最精確

🔥 最後總結（考前30秒）

👉 Selection → n^2
👉 Binary → log n
👉 Recursion → base case
👉 ADT → spec ≠ implementation
👉 Big-O → 看最大項

如果你要更狠一點👇
👉「🔥老師會出的選擇題版本（陷阱超多）」
👉「🔥直接猜期中考題（命中率很高）」
👉「🔥幫你做一張A4 cheat sheet」

直接說 👍